

图 6.5　图 6.4 的程序所声明变量的存储单元

6.2　全局栈式存储分配

上节介绍的是单个活动记录内数据的布局，本节介绍程序运行时所有活动记录在存储空间的组织，并描述过程的目标代码怎样访问绑定到局部名字的存储单元。本节所讨论的栈式存储分配可用于 C、FORTRAN 和 Pascal 等面向过程的语言，也可用于 Java 和 C++等面向对象的语言。

6.2.1　运行时内存的划分

在逻辑地址空间，一个程序运行时所用空间像图 6.6 那样，由程序区和若干个数据区组成。例如，在 Linux 操作系统上，C 编译器就是按这种方式将内存细分的。

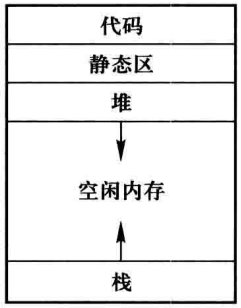


图 6.6　运行时内存空间的划分

目标代码的长度在编译时即可确定，并且运行时不会改变，通常安排在内存的低地址区。一些程序数据，例如全局常量，还有编译器产生的数据，例如支持垃圾收集的信息，它们的存储大小在编译时都可以确定，因此把它们安排在静态确定的数据区中。尽可能把数据安排在静态区的一个理由是，这些数据的地址可以编译到目标代码中去，以提高运行时对这些数据的访问速度。在 FORTRAN 语言的早期版本中，所有数据都可以静态分配，因为它们不允许递归过程。

为了在运行时充分利用存储空间，栈和堆一般安排在地址空间剩余部分的两端。它们是动态的，在程序运行过程中它们占用的存储空间不断变化，相向增长。在实际机器上，栈向低地址方向增长，堆向高地址方向增长。

像 C、FORTRAN 和 Pascal 这样的语言，由于过程递归的次数一般不是静态可确定的，因此活动记录不可能静态分配，虽然每个活动记录的大小可以静态确定。另一方面，由于一个过程活动终止时其活动记录不再需要，又由于过程活动的生存期要么嵌套，要么无重叠，因此可以把当前所有存活过程活动的活动记录组织成一个栈。这种存储分配策略从 6.2.2 节开始讨论。

许多编程语言允许程序员在程序控制下分配和释放存储块，如 C 语言的函数 malloc 和 free 用于此目的。这些存储块中数据对象的生存期可能比要求分配这些存储块的过程更长，因此它们的生存期不遵守栈式规则。堆用来管理这类长命数据。数据放在栈上比放在堆上的开销要小一些，这是由它们对数据的分配和回收方式决定的，6.5 节专门讨论堆管理。

第 10 章还会对运行时内存空间的组织做更细一点的介绍。

6.2.2 活动树和运行栈

6.2.1 节已经提到，对于常见的面向过程的语言，如果 a 和 b 都是过程活动，那么它们的生存期或者嵌套，或者无重叠。也就是说，如果控制在离开 a 之前进入 b，那么控制在离开 b 之后才能离开 a。由此可知，可以用树来描绘控制进入和离开活动的次序，这样的树称为**活动树**。在活动树中：

（1）每个结点代表一个过程活动；

（2）根结点代表主过程的活动；

（3）结点 a 是结点 b 的父结点，当且仅当控制流从活动 a 进入活动 b；

（4）结点 a 处于结点 b 的左边，当且仅当活动 a 的生存期先于活动 b 的生存期。

因为结点和活动一一对应，所以当控制处于某结点代表的活动时，就直接说控制处于这个结点。

例 6.1 图 6.1 程序的活动树画在图 6.7 中，其中过程名都用它的第一个字母表示，括号中的数据是活动的实参。程序的控制流对应于活动树的后根遍历。 □

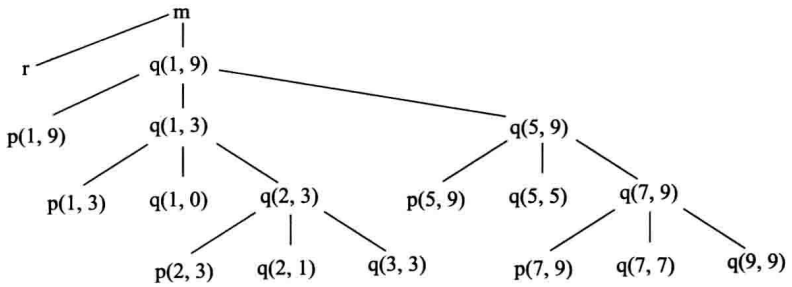


图 6.7 图 6.1 程序的活动树

例 6.2 图 6.8 给出了控制进入 q(2,3)代表的活动时，已经执行完毕和正在执行的活动。其中已经执行完毕的活动在虚线的下端，正在执行、尚未结束的活动在根结点到结点 q(2,3)的这条实线路径上。尚未开始的活动在图中没有画出。 □

从图 6.8 可以看出，当前活跃的过程活动可以保存在一个栈中。当活动开始时，把这个活动的结点压入栈中，当它结束时，把它的结点从栈中弹出。这样的栈被称为**控制栈**。对于图 6.8 的情况，控制栈的内容从栈底到栈顶依次为：

m,q(1,9),q(1,3),q(2,3)

如果控制栈中的信息包含过程活动的活动记录，控制栈就成了活动记录栈，通常称之为**运行栈**。当一个过程被调用时，它的一个新活动记录被压入栈，局部变量被绑定到其中的存储单元；当对应这次调用的活动终止时，该活动记录被退栈，局部变量不再绑定到其中的存储单元，恢复原来的绑定（如果有的话）。由于一个过程的每次调用都会引起一个新的活动记录进栈，所以过程的每次活动都会把局部变量绑定到新的存储单元。

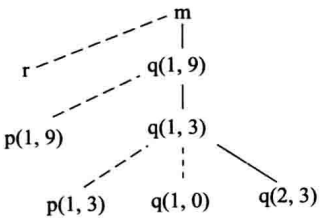


图 6.8　控制栈包含实线上的结点

例 6.3　图 6.9 表示当控制流通过如图 6.7 所示的活动树时，活动记录压入运行栈和从运行栈退出的几个瞬像（snapshot，也称快照），树上的虚线仍然引向已经结束的活动。注意，因为数组 a 是全局的，它分配在静态区，因此没有出现在该图上。程序执行开始时有过程 m 的活动，当控制到达 m 体中第一个调用时，过程 r 被激活，它的活动记录被压入栈。当控制从该活动返回时，栈顶活动记录被退栈，栈中仅剩下 m 的活动记录。在 m 的继续执行过程中，当控制到达以 1 和 9 为实参的对过程 q 的调用时，q 的一个活动记录被压入栈。

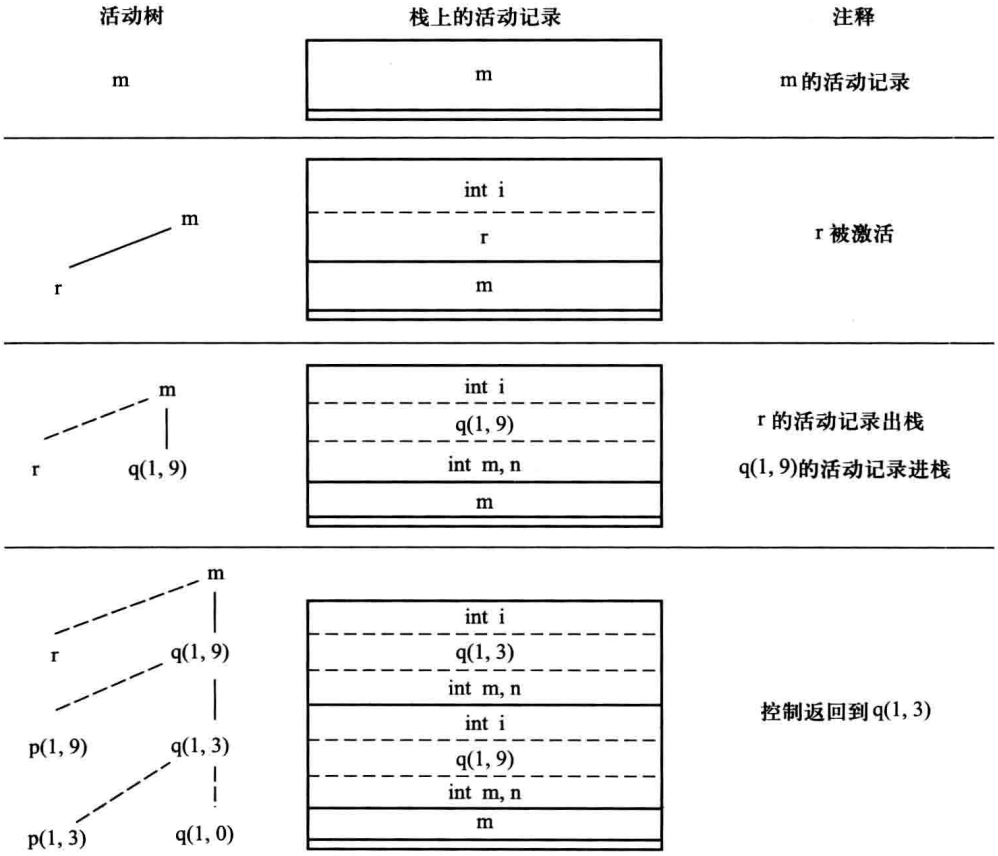


图 6.9　活动记录在栈中的分配（栈向上长）

图6.9的最后两个瞬像之间有多个活动发生。在最后一个瞬像中，活动p(1,3)和q(1,0)已经开始并终止在q(1,3)的生存期里中，它们的活动记录也都已经压栈并退栈，因此只留下q(1,3)的活动记录在栈顶。 □

6.2.3 调用序列

从先前的介绍可以知道，过程调用和过程返回都需要执行一些代码来管理运行栈：分配和回收活动记录、保存或恢复机器状态等。在过程调用时执行的分配活动记录，以及把信息填入其域中的代码被称为**过程调用序列**；而在过程返回时执行的恢复机器状态，回收活动记录，以及让调用过程继续执行的代码被称为**过程返回序列**。

即使是同一种语言，过程调用序列、过程返回序列和活动记录的布局也会因实现而异。过程调用序列的代码常常分成两部分，分别位于调用过程和被调用过程中。过程调用序列在这两者之间的划分也不是唯一的。源语言、目标机器和操作系统强加的约束可能使得某种方法比另一些方法更合适。过程返回序列也是如此。

下面一些原则对设计调用序列、返回序列和活动记录布局是有益的。

(1) 调用者和被调用者之间交流的数据一般放在被调用者活动记录的开始处，尽可能靠近调用者的活动记录。这样做便于调用者把实参的值放到自身活动记录的顶端，而不需要此刻就建立被调用者完整的活动记录，甚至不用知道它的布局。这样做还允许使用参数个数可变的过程，如C语言的标准库函数printf，这个问题后面再讨论。被调用者也知道返回值应该放在靠近调用者活动记录的地方，不管有多少个实参，它们都正好在返回值的下面。

(2) 固定长度的项通常放在活动记录的中间。它们一般包括控制链、访问链和机器状态域。如果每次调用时正好保存机器状态中同样的成分，那么可以执行同样的代码来完成机器状态的保存和恢复。而且，如果机器状态信息是标准化的，那么出现错误时，调试器等程序可以很轻松地解读运行栈的内容。

(3) 对编译时不能及时知道其大小的一些项，放在活动记录的末端。大多数局部变量有固定的长度，这些长度能够由编译器在检查变量类型时确定。但是，有些局部变量的大小在程序运行前一直不能确定，最常见的例子是动态大小的数组(其大小依赖于过程参数的数组)。还有，所需的临时数据空间的大小通常依赖于代码生成阶段怎样尽可能地把临时数据存放在寄存器中。因此，虽然临时数据空间的大小在编译时最终可以知道，但它可能不是在中间代码生成时就知道的。

作为一个例子，图6.10给出一种调用者和被调用者怎样协作管理栈的建议(无须访问链的语言)。寄存器top_sp指向栈顶活动记录的末端；另一个寄存器base_sp指向栈顶活动记录中控制链所在的位置，它作为访问栈顶活动记录中内容的基地址。假定过程p调用过程q，调用序列如下。

(1) p计算实参，依次将它们压入栈顶，也就是放到q的活动记录中，并在栈顶上留出放返

回值的空间。在这些操作过程中,top_sp 的值在不断减小。

(2) p 把返回地址压入 q 的活动记录,把控制转到 q。

(3) q 将要保存的寄存器(base_sp 除外)的值和其他机器状态信息压入栈,然后把 base_sp 的当前值(p 的 base_sp)压入栈,并把当前 top_sp 的值作为自己 base_sp 的值。

(4) q 根据局部数据域和临时数据域的大小来减小 top_sp 的值,也就是进行局部数据和临时数据的空间分配,并初始化它的局部数据,开始执行过程体,如图 6.10 所示。

在上述第(3)步中,把 p 的 base_sp 压入栈就是建立控制链,由于控制链是按运行时过程调用关系从被调用者的活动记录指向调用者的活动记录,因此控制链也称为**动态链**。

返回序列如下。

(1) q 把返回值置入活动记录中存放返回值的地方。

(2) 与调用序列的步骤(4)相对应,q 增加 top_sp 的值。

(3) q 恢复寄存器(包括 base_sp)和机器状态,把控制转到 p。

(4) 与调用序列的步骤(1)相对应,p 增加 top_sp 的值,并取出返回值。

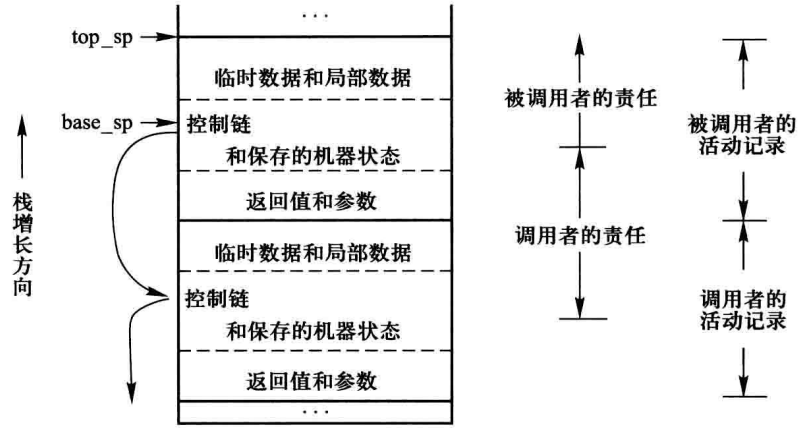


图 6.10 调用者和被调用者之间的任务划分

例 6.4 一个 C 语言函数如下:

```
func(i)long i;{
    long j;
    j=i-1;
    func(j);
}
```

为使生成的汇编代码尽量少,用了这个不会终止的递归函数。在 x86/Linux 系统上用版本标识为 GCC:(GNU)egcs-2.91.66 19990314/Linux(egcs-1.1.2 release)的 GCC 编译器,为该函数生成的汇编代码(略去与问题无关的部分)及添加的注释如下:

```
func:
    pushl  %ebp                 //将老的基地址指针压栈
    movl  %esp,%ebp             //将当前栈顶指针作为基地址指针
    subl  $4,%esp               //为局部变量 j 分配空间
    movl 8(%ebp),%edx           //取形参 i 的值到寄存器,i 的地址是 8(%ebp)
    decl  %edx                  //i-1
    movl  %edx,-4(%ebp)         //i-1 的值赋给 j,分配给 j 的地址是-4(%ebp)
    movl-4(%ebp),%eax           //和下一条指令一起,完成将实参 j 的值压栈
    pushl  %eax
    call  func                  //函数调用,将返回地址压栈,修改程序计数器
    addl  $4,%esp               //栈顶指针恢复到参数压栈前的位置
.L1:
    leave                       //相当于 movl %ebp,%esp 和 popl %ebp
    ret                         //相当于 popl %eip,将 esp 恢复到调用前参数压栈后
                                //的位置,返回调用者。eip 存放下步执行指令的地址
```

从上面的汇编代码可以分析出该函数一个活动记录的内容,如图 6.11 所示。其中 esp 是栈顶指针寄存器(执行 movl 8(%ebp),%edx 指令时栈顶指针所指的位置就是图中所标明的位置),ebp 是基地址寄存器。

在这个例子中,调用序列的代码在调用者的部分是 pushl %eax 和 call func,在被调用者的部分是 pushl %ebp、movl %esp,%ebp 和 subl $4,%esp。返回序列的代码在被调用者的部分是 leave 和 ret,在调用者的部分是 addl $4,%esp。

注意,现在流行的较高版本 GCC 编译器,所分配的局部数据空间比局部变量的实际需求略大一些,一个原因是为了栈对齐,另一个原因是预留一点空间。有兴趣的读者可以参考相关资料。还需要注意一点,为提高参数传递的效率,不是用 push 指令将参数逐个压栈,而是先修改 esp 的值,空出所需的参数区,然后用 movl 指令将实参逐个进栈。 □

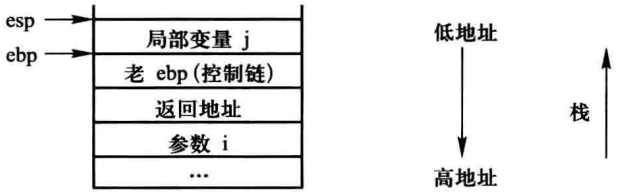


图 6.11 活动记录的内容及相关信息

上面的调用序列和返回序列可用于过程的参数个数可变的情况,以 C 语言的标准库函数 printf 为例。在一个程序中,每次 printf 调用的实参个数是清楚的,编译器产生将这些实参逆序压栈的代码,被调用函数 printf 虽然不知道参数的个数,但是它能准确地知道第一个参数的位置。

因此 printf 的实现首先取第一个参数——格式控制字符串，然后分析它的格式控制要求，根据格式控制中的格式说明，到栈中取第二、第三个参数等。

6.2.4 栈上可变长度数据

在现代编程语言中，通常把编译时不能确定所需空间的数据分配到堆上。但是有些情况下把数组、对象和其他未知大小的结构分配到栈上也是可能的，尽可能把它们分配在栈上而不是堆上的理由是避免垃圾收集器收集它们的开销。注意，只有那些局部于一个过程并在该过程返回后不可访问的数据对象才能分配到栈上。本节以动态数组为例，介绍编译时如何将不能确定所需空间的数据分配到栈上，该方案也可用于局部于过程并且大小依赖于过程参数的其他类型的数据。

在图 6.12 中，局部数组的大小要等到过程激活时才能确定。过程 p 有两个不能静态确定大小的局部数组，在编译时，在活动记录中为这两个数组分别分配一个存放数组指针的单元。运行时，这两个数组的大小能确定后，在栈顶为这些数组分配空间，并把起始地址置入存放数组指针的单元。这样对这些数组的访问是通过活动记录中的数组指针间接进行的。

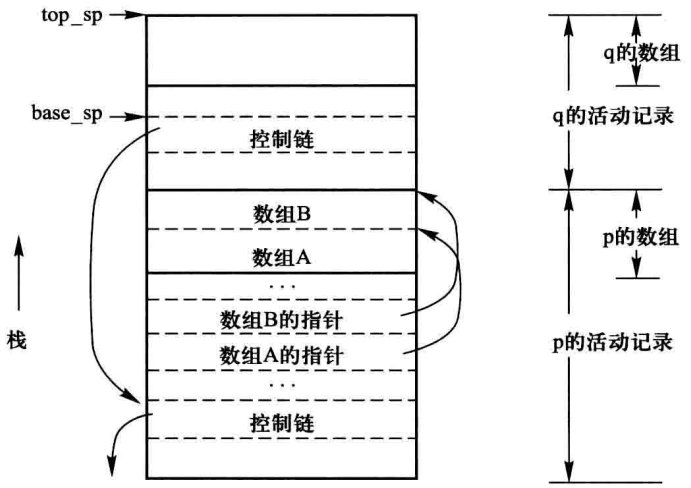


图 6.12 访问动态分配的数组

6.2.5 悬空引用

只要存储空间可以回收，就有可能出现悬空引用问题。引用某个已被回收的存储单元就称为**悬空引用**(dangling reference)。使用悬空引用是一种逻辑错误，因为按大多数语言的语义，已

被回收的存储单元的值是没有定义的。更糟糕的是，已被回收的存储单元可能随后被分配用来存放其他数据，因此有悬空引用错误的程序会出现难以理解的、不会被捕获的错误。悬空引用在栈上和堆上都有可能出现，下面是一个在栈上出现悬空引用的例子。

例 6.5 在图 6.13 的 C 语言程序中，过程 dangle 返回一个指向绑定到局部名 j 的存储单元的指针。当控制从 dangle 返回到 main 时，dangle 的活动记录已经回收，并可能已另有安排。因为 main 中 q 的值是这个存储单元的地址，q 被称为**悬空指针**，对 q 指向的对象的访问就是一个悬空引用。 □

```
int * dangle( ){
    int  j=20;
    return  &j;
}
main( ){
    int  * q;
    q=dangle( );
}
```

图 6.13 q 指向已经回收了的存储单元

6.3 非局部名字的访问

语言的作用域规则规定了如何处理非局部名字的访问。一种常用的规则是**词法作用域**或**静态作用域**规则，它仅根据程序正文静态地确定适用于一个名字的某个引用性出现的该名字的声明(简单说为用于名字的声明)。许多语言，如 C、Java 和 Pascal 等，都使用静态作用域规则。本节首先考虑 C 语言那样的非局部名字。由于 C 语言不允许嵌套的过程声明，因此所有的非局部名字都可以静态地绑定到所分配的存储单元。

像 Pascal 和 ML 等语言，它们允许过程嵌套，并使用静态作用域，确定用于名字的声明需要根据过程的嵌套层次来决定。和 C 语言不同的是，Pascal 语言的非局部名字不一定就是全局名字。当运行时访问非局部名字，首先要确定该非局部名字被绑定到的活动记录，本节讨论利用访问链寻找该活动记录的一种方法。

另一种作用域规则是动态作用域规则，它是在运行时根据当前存活的过程活动来确定用于名字的声明。使用动态作用域的语言很少，如 LISP。作为比较，6.3.3 节将讨论动态作用域的实现。